

Inverted Pendulum

Systems Engineering – Team 05

Equations of Motion

Non-linear Dynamics (Lagrangian Approach):

We derived the system's "True Physics" using the **Euler-Lagrange** method to capture real-world non-linearities like centripetal forces

Translation: $(M + m)\ddot{x} + mL\ddot{\theta} \cos(\theta) - mL\dot{\theta}^2 \sin(\theta) = F - b\dot{x}$

Rotation: $mL^2\ddot{\theta} + mL\ddot{x} \cos(\theta) - mgL \sin(\theta) = -c\dot{\theta}$

Code Implementation:

In our Python implementation, we utilize a **Mass Matrix (A)** and a **Force Vector (B)** to solve for accelerations ($\ddot{x}, \ddot{\theta}$) at every time step.

State Representation: The system state is tracked as a 4D vector: $\mathbf{x} = [x, \dot{x}, \theta, \dot{\theta}]^T$

```
x, x_dot, theta, theta_dot = state

# Mass matrix
A = np.array([
    [self.M + self.m, self.m * self.L * np.cos(theta)],
    [self.m * self.L * np.cos(theta), self.m * self.L**2]
])

# Force vector (pendulum_torque added to rotational equation)
B = np.array([
    force - self.b * x_dot + self.m * self.L * theta_dot**2 * np.sin(theta),
    -self.c * theta_dot + self.m * self.g * self.L * np.sin(theta) + pendulum_torque
])

# Solve for accelerations
x_ddot, theta_ddot = np.linalg.solve(A, B)

return np.array([x_dot, x_ddot, theta_dot, theta_ddot])
```

Controller Design - PID

Dual Loop Structure

To achieve the simultaneous goals of pendulum stabilization and cart positioning (Target $x = 2\text{m}$), we implemented a cascaded structure :

- Outer Loop (Position): Translates cart displacement error into a "Desired Tilt Angle" (θ_{set}).
- Inner Loop (Angle): Processes the tilt error to generate the corrective motor force (F).

On Simulation:

- Integral Anti-Windup: Both loops use `np.clip()` on integral terms to prevent overshoot and saturation lag (Outer: ± 10 , Inner: ± 100).

```
x, x_dot, theta, theta_dot = state

# Time step
if self.previous_time is None:
    self.previous_time = t
    dt = 0.02
else:
    dt = max(t - self.previous_time, 0.02)

# Outer loop: position PID → desired angle
pos_error = self.x_target - x
self.pos_integral = np.clip(self.pos_integral + pos_error * dt, -10.0, 10.0)
angle_setpoint = (self.kp_pos * pos_error +
                  self.ki_pos * self.pos_integral +
                  self.kd_pos * x_dot)
angle_setpoint = np.clip(angle_setpoint, -self.max_angle_setpoint, self.max_angle_setpoint)

# Inner loop: angle PID
angle_error = theta - angle_setpoint

# PID terms
P = self.kp * angle_error
self.integral = np.clip(self.integral + angle_error * dt, -100.0, 100.0)
I = self.ki * self.integral
D = self.kd * theta_dot

self.previous_time = t
force = np.clip(P + I + D, -100.0, 100.0)
return force
```

Controller Design - LQR

Optimal State-Feedback Control Strategy

Unlike our PID controller, which uses a dual loop structure, the LQR uses a **single unified** controller.

Target state: $x = [2 \ 0 \ 0 \ 0]^T$

First the dynamics are linearised (state $x = [x \ \dot{x} \ \theta \ \dot{\theta}]$) into the form $\dot{x} = A x + B u$.

Then, the quadratic **cost function** is minimised.

Control Law

The optimal feedback gain K is then computed by solving the CARE equation. The control force is then calculated as:

$$u = -K(x - x_{\text{setpoint}})$$

where $K = [k_x, k_{\dot{x}}, k_{\theta}, k_{\dot{\theta}}]$

Therefore, all 4 states (position, velocity, angle, angular velocity) **contribute** to the control force **simultaneously**.

```
"""Linearize around upright equilibrium."""
M, m, L, b, c, g = self.M, self.m, self.L, self.b, self.c, self.g
M_total = M + m

A = np.array([
    [0, 1, 0, 0],
    [0, -b/M_total, -m*g/M_total, 0],
    [0, 0, 0, 1],
    [0, b/(L*M_total), g*(M+m)/(L*M_total), -c/(m*L**2)]
])

B = np.array([[0], [1/M_total], [0], [-1/(L*M_total)]])
return A, B
```

```
"""Solve CARE for optimal gain K."""
P = solve_continuous_are(self.A, self.B, self.Q, self.R)
return np.linalg.inv(self.R) @ self.B.T @ P
```

```
"""Compute control force: u = -K(x - x_setpoint)."""
error = state - self.setpoint
force = -float(self.K @ error.reshape(-1, 1))
return np.clip(force, -100.0, 100.0)
```

Controller Design – Pole Placement

Optimised State-Feedback Control Strategy

Rather than optimising a cost function like LQR, pole placement **directly assigns** the closed loop dynamics of the system. The rest of the initial setup for this design is the **same as LQR**: same target x and linearising the state.

Control Law

Rather than solving the CARE equation, the gain matrix K is calculated such that $\text{eig}(A-BK) = \text{desired poles}$

It then uses the same logic as LQR to compute the control force, therefore all **states contribute** to the force **simultaneously**.

```
"""Linearize around upright equilibrium."""
M, m, L, b, c, g = self.M, self.m, self.L, self.b, self.c, self.g
M_total = M + m

A = np.array([
    [0, 1, 0, 0],
    [0, -b/M_total, -m*g/M_total, 0],
    [0, 0, 0, 1],
    [0, b/(L*M_total), g*(M+m)/(L*M_total), -c/(m*L**2)]
])

B = np.array([[0], [1/M_total], [0], [-1/(L*M_total)]])
return A, B
```

```
"""Compute K such that eig(A-BK) = desired_poles."""
result = place_poles(self.A, self.B, self.desired_poles)
return result.gain_matrix
```

```
"""Compute control force: u = -K(x - x_setpoint)."""
error = state - self.setpoint
force = -float(self.K @ error.reshape(-1, 1))
return np.clip(force, -100.0, 100.0)
```

Sensor Noise and Hardware Realism

Our noise was generated in simulation to simulate real life behaviour:

Position noise: $\sigma = 0.005$ m (5 mm)

Angle noise: $\sigma = 0.01$ rad

The noise was filtered through a **low-pass filter** then differentiated to velocities. This generates smoothed signal rather than the noisy input, eliminating 'spiky' artifacts ensuring accurate state estimation.

Stage 1 – Low-pass filter:

$$y[k] = \alpha y[k-1] + (1 - \alpha)x[k], \quad \alpha = \frac{\tau}{\tau + T_s} \quad (2)$$

Stage 2 – Dirty derivative:

$$\dot{y}[k] = \frac{y[k] - y[k-1]}{T_s} \quad (3)$$

Parameters: $\tau_x = 0.1$ s, $\tau_\theta = 0.08$ s, $T_s = 0.02$ s.

Authentic Motor Specs:

Uses real-world **LP 12V** motor parameters, including **13.33 Ohm** resistance and a **0.363 Nm** stall torque.

Voltage Saturation:

Implements strict **1.0V to 12.0V** limits to mirror physical battery and driver constraints.

Dynamic Torque Calculation:

Torque is derived from the actual current (I), which fluctuates based on voltage and velocity rather than being a constant value.

Real-World Friction:

Includes viscous damping for the cart (0.1 N/m/s) and rotational damping at the pivot (0.01 Nm/rad/s).

Mass Coupling:

The pendulum's weight and motion actively impact the cart's acceleration, reflecting true Newtonian physics.

Physical Constraints:

All movements account for a 30mm wheel radius and specific mass distributions.

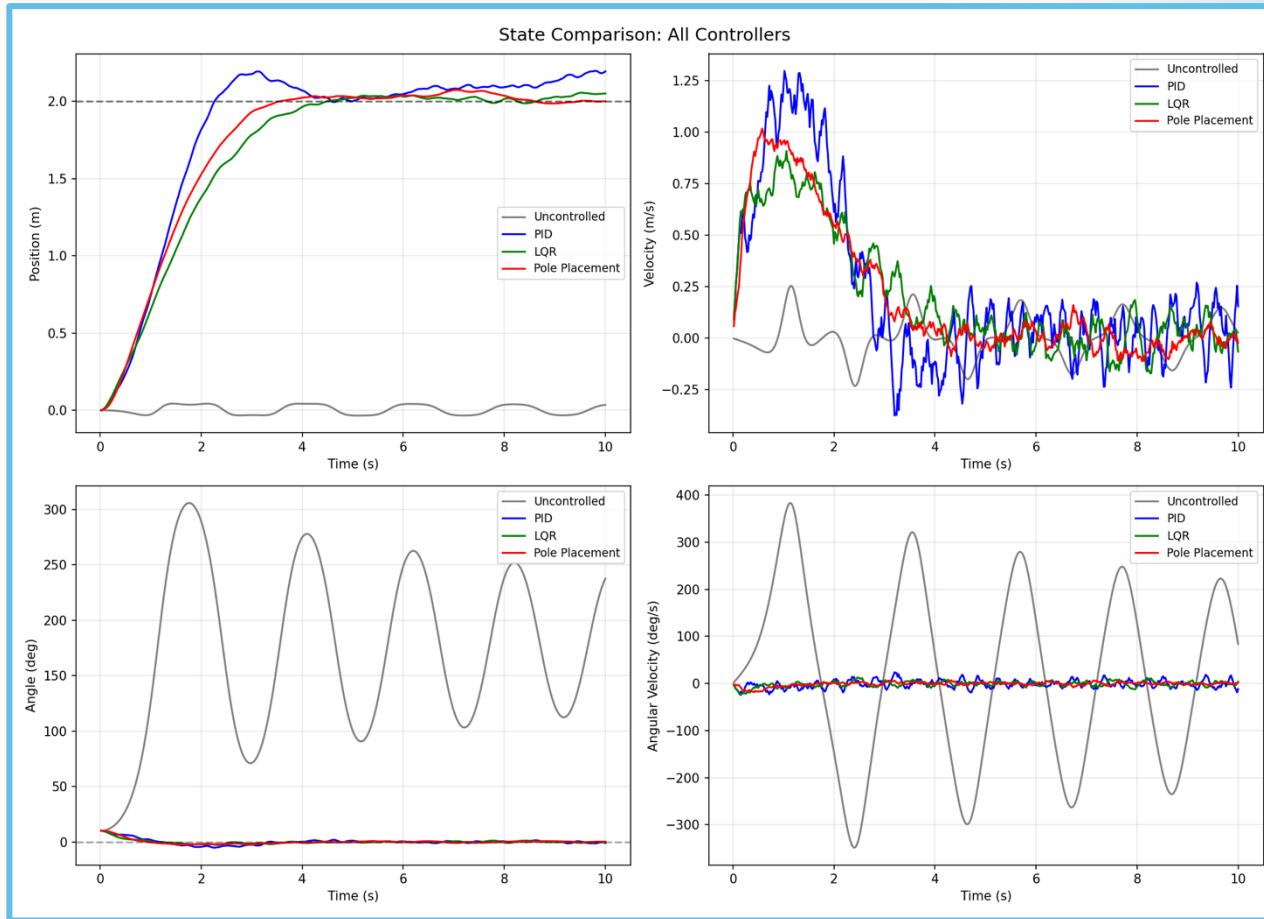
Gaussian Sensor Noise:

Simulates real-world "jitter" with a 5mm position deviation and a 0.57-degree angle noise.

50 Hz Update Rate:

The system operates on a 20ms sample period, the industry standard for embedded systems.

Batch Comparison



PID:

Rise time (90%): 2.36s
Max position: 2.257m
Overshoot: 0.257m (12.9%)
Final position: 2.166m
Oscillation (2s): 0.072m
Settling time (5%): N/A

LQR:

Rise time (90%): 3.54s
Max position: 2.050m
Overshoot: 0.050m (2.5%)
Final position: 2.021m
Oscillation (2s): 0.027m
Settling time (5%): 4.10s

Pole Placement:

Rise time (90%): 3.18s
Max position: 2.048m
Overshoot: 0.048m (2.4%)
Final position: 1.992m
Oscillation (2s): 0.048m
Settling time (5%): 3.60s

Performance Testing

Max Initial Angle Test

- Uses binary search between 5° and 60° to find the largest starting tilt the controller can recover from
- Runs 10-second simulation with noise and motor constraints
- Pass criteria: Pendulum stays within $\pm 90^\circ$ throughout AND ends within $\pm 30^\circ$

Max Disturbance Test

- Starts with small 5° tilt, applies an impulse force at $t=2s$
- Binary searches between 0-100N to find maximum survivable push
- Simulates real-world bumps/collisions the system might experience

ROBUSTNESS TEST RESULTS		
Controller	Max Initial Angle	Max Disturbance
PID	21.3°	55.1 N
LQR	20.5°	59.0 N
Pole Placement	20.5°	57.0 N